

Project Overview: ReviewPulse AI

ReviewPulse AI is an intelligent e-commerce sentiment analysis and customer feedback dashboard designed to help sellers and businesses transform messy customer reviews into structured, actionable insights instantly.

Tech Stack & Dependencies

The project relies on a modern Python stack optimized for web-based AI applications and data visualization:

- **Frontend & UI:** streamlit for building interactive, browser-based web interfaces.
- **LLM Orchestration:** langchain, langchain-openai, and langchain-core for managing conversational history and communicating with OpenAI models.
- **Data Processing & Utilities:** pandas, textblob, and requests for handling data formatting and text processing.
- **Data Visualization:** plotly and matplotlib for generating visual analytics and feedback trends.

Core Architecture & Code Breakdown (app.py)

1. Page Configuration & Session Management

The application initializes with a centered Streamlit layout and establishes persistent session state variables to track user authentication, secure API keys, and message history:

- `st.session_state["api_authenticated"]`: Tracks whether the user has unlocked the system.
- `st.session_state["api_key"]`: Stores the user-provided OpenAI API key securely during the session.
- `st.session_state["messages"]`: Maintains historical chat messages and system prompts.

2. Secure Access Portal (Lock Screen)

Before loading the main dashboard, the application enforces a password-masked API key gate:

- Users input their OpenAI API key into a secure password field (`type="password"`).
- Clicking **Unlock Dashboard** validates that the key is non-empty, saves it to session state, and triggers `st.rerun()` to load the primary interface.

3. Expert E-commerce Analyst System Prompt

Once unlocked, the application injects a specialized system role using LangChain's `SystemMessage`:

- **Role:** Expert e-commerce data analyst and sentiment specialist.
- **Output Structure:** Analyzes reviews to generate a structured report featuring an Overall Satisfaction Score (out of 10), Top 3 positive aspects (Pros), Top 3 negative aspects (Cons), and brief actionable seller recommendations.

4. Interactive Chat Loop

- Utilizes gpt-4o-mini with a temperature of 0.3 for consistent, reliable analytical outputs.
- Automatically pins the user input box to the bottom center using Streamlit's native `st.chat_input()`.
- Wraps AI generation inside an active `st.spinner("Analyzing customer review patterns...")` element to provide visual loading feedback.



Deployment & Cloud Troubleshooting

Deploying Streamlit applications to **Streamlit Community Cloud** requires a precise configuration:

1. **Repository Structure:** Ensure your GitHub repository contains `app.py` directly in the root directory alongside `requirements.txt` and `README.md`.
2. **Dependency Synchronization:** Every package imported in `app.py` (specifically `langchain`, `langchain-openai`, and `langchain-core`) **must** be explicitly declared inside `requirements.txt`. Missing entries result in `ModuleNotFoundError` during cloud builds.
3. **App Rebooting:** If dependency errors occur post-deployment, updating `requirements.txt` on GitHub and selecting **Reboot app** from the Streamlit Cloud management panel forces a clean package re-installation.